

IoTPlatform DotNet 项目学习指南

1. 项目一句话概览

这是一个基于 **.NET 8 + Furion + SqlSugar** 的工业互联网平台后端，整体分成：

- 1. **架构核心层**：提供基础能力、数据库访问、MQTT、TDengine、第三方集成等。
- 2. **应用模块层**：按业务域拆分，例如系统管理、可视化开发、任务流、物模型、设备驱动。
- 3. **应用服务层**：对外提供 Web API、实时通信、gRPC 采集服务，以及运行时宿主。

从代码组织看，仓库已经天然按“**核心能力 -> 业务模块 -> 运行服务**”分层。

2. 整体架构

2.1 目录分层

目录	作用	关键内容
01-架构 核心	平台 底座	IotPlatform.Core、Extras.DatabaseAccessor.SqlSugar、Extras.MQTT、Extras.TDengine
02-应用 模块	业务 模块	系统、任务、物联、可视化开发、脚本引擎、设备驱动等
03-应用 服务	运行 宿主	IotPlatform Web 主站、IotPlatform.Web.Core 启动配置、IotPlatform.CollectionService 采集服务

2.2 启动链路

主 Web 服务入口：

- 03-应用服务\IotPlatform\Program.cs
- 03-应用服务\IotPlatform.Web.Core\Startup.cs

运行流程可以概括为：

- 1. Program.cs 调用 Serve.Run(...) 启动 Furion Web 组件。
- 2. Startup.cs 注册项目核心能力：
 - 配置中心
 - SqlSugar
 - JWT 认证
 - CORS
 - EventBus
 - SignalR
 - gRPC
 - 日志
 - 多个 HostedService
- 3. 各业务模块里的 IDynamicApiController 服务会被暴露为动态 API。

2.3 运行时组成

服务	位置	作用
Web 主服务	03-应用服务\IotPlatform	对外 API、鉴权、业务逻辑入口、SignalR、gRPC 接入
Web 核心配置	03-应用服务\IotPlatform.Web.Core	注册所有中间件、配置项、缓存、调度、事件总线
采集服务	03-应用服务 \IotPlatform.CollectionService	设备采集、Redis 缓存、变量上报、运行时 gRPC 通信

当前代码里，**Web 主服务**和**采集服务**是两个独立宿主：

- Web 服务更偏业务编排和平台管理；
- CollectionService 更偏设备接入、变量采集、数据上报。

3. 关键技术栈

技术	在项目中的定位
.NET 8	整体运行时
Furion	应用启动、动态 API、依赖注入、统一返回、基础框架能力
SqlSugar	ORM 与多库访问
JWT	API 认证鉴权
Redis / NewLife.Redis	缓存、消息、运行时数据同步
SignalR	实时消息推送
gRPC	Web 服务与采集服务通信
Serilog	采集服务日志
TDengine	时序数据归档
MQTT	设备/消息相关能力

4. 各模块功能说明

4.1 架构核心层 01-架构核心

模块	说明
IotPlatform.Core	核心基础库，封装项目公共能力和基础扩展，是大多数业务模块的底座
Extras.DatabaseAccessor.SqlSugar	SqlSugar 扩展与数据库访问支持
Extras.MQTT	MQTT 相关能力

模块	说明
<code>Extras.TDengine</code>	TDengine 相关能力
<code>Extras.Thridparty</code>	第三方能力封装

4.2 应用模块层 02-应用模块

分组	说明
<code>00-Common</code>	公共 DTO、通用实体、基础业务共用代码
<code>01-0Auth</code>	认证授权相关
<code>02-System</code>	系统管理能力，如菜单、权限、用户、组织等基础设施
<code>03-BusApp</code>	平台业务应用层能力
<code>04-DataWeaving</code>	数据编织/数据处理相关
<code>05-Message</code>	消息与策略消息能力
<code>06-Task</code>	任务域，包括定时任务、工作流、程序块
<code>07-Thing</code>	物联业务域，包括物模型、告警、远程控制、视频设备、OEE 统计
<code>08-VisualData</code>	可视化数据展示相关
<code>09-Engine</code>	运行引擎层，包括脚本引擎、MQTT 引擎、可视化开发引擎实体
<code>10-VisualDev</code>	在线可视化开发/表单设计/列表设计/运行时数据处理
<code>11-Extend</code>	扩展功能，如 License 等
<code>13-Device</code>	设备驱动体系、采集契约、驱动接口与具体协议实现

4.3 重点业务子模块

09-Engine

模块	作用
<code>Engine.Entity</code>	引擎相关实体定义
<code>JsScript.Engine</code>	脚本引擎、内部变量同步、运行时变量桥接
<code>Mqtt.Engine</code>	MQTT 引擎能力
<code>VisualDev.Engine</code>	可视化开发运行时依赖的引擎模型/解析能力

10-VisualDev

模块	作用
<code>VisualDev</code>	动态 API 服务实现，负责功能设计、发布、运行时数据接口

模块	作用
<code>VisualDev.Entity</code>	功能设计实体、发布实体、分类实体、Portal 实体、DTO
<code>VisualDev.Interface</code>	<code>IVisualDevService</code> 、 <code>IRunService</code> 等抽象接口

13-Device

模块	作用
<code>IotPlatform.Driver</code>	驱动管理与驱动核心能力
<code>IotPlatform.Driver.Interface</code>	驱动接口抽象
<code>IotPlatform.Driver.Entity</code>	驱动相关实体
<code>Driver\Siemens / Omron / Modbus / Melsec</code>	具体 PLC / 协议驱动实现
<code>IotPlatform.Collection.Contracts</code>	采集服务通信契约

5. 推荐学习路径

5.1 快速入门（先建立全局认知）

建议按下面顺序阅读：

1. `IotPlatform.sln`
 - 先看解决方案分组，建立模块地图。
2. `03-应用服务\IotPlatform\Program.cs`
 - 看主服务怎么启动。
3. `03-应用服务\IotPlatform.Web.Core\Startup.cs`
 - 看项目到底注册了哪些框架能力。
4. `03-应用服务\IotPlatform.Web.Core\IotPlatform.Web.Core.csproj`
 - 看 Web 宿主依赖了哪些业务模块。
5. `02-应用模块\10-VisualDev`
 - 这是理解“低代码表单 + 运行时数据”的关键模块。
6. `03-应用服务\IotPlatform.CollectionService\Program.cs`
 - 看设备采集链路如何单独部署。

5.2 提高阶段（按链路理解系统）

建议从“请求如何走到数据库/设备”这个角度继续：

1. **Web API 链路**
 - `Program.cs` -> `Startup.cs` -> 动态 API Service -> Repository / RunService -> DB
2. **表单运行时链路**
 - `VisualDevModelDataService` -> `RunService` -> 动态 SQL / 表结构解析 -> 数据库
3. **设备采集链路**
 - `CollectionService` -> Driver -> Redis/Reporter -> Web gRPC/缓存
4. **实时推送链路**
 - 变量变化 -> HostedService -> SignalR / Redis / TDengine

6. VisualDev（表单/低代码）模块重点说明

这是你后续改“表单编辑相关接口”最需要熟悉的模块。

6.1 核心实体

设计态实体

- 02-应用模块\10-VisualDev\VisualDev.Entity\Entity\VisualDevEntity.cs

主要字段：

- FormData：表单 JSON 配置
- ColumnData：列表 JSON 配置
- Tables：关联表配置
- DbLinkId：数据库连接
- WebType：页面类型
- EnableFlow：是否启用流程

这张表对应 BASE_VISUAL_DEV，可以理解为草稿/设计态定义。

发布态实体

- 02-应用模块\10-VisualDev\VisualDev.Entity\Entity\VisualDevReleaseEntity.cs

这张表对应 BASE_VISUAL_RELEASE，可以理解为运行时发布版本。

一个非常关键的认识：

- 设计器改的是 VisualDevEntity
- 运行时表单通常取的是 VisualDevReleaseEntity

6.2 核心服务

1. VisualDevService

文件：

- 02-应用模块\10-VisualDev\VisualDev\VisualDevService.cs

职责：

- 功能设计的增删改查
- 发布/取消发布
- 模板字段同步
- 菜单生成
- 设计态与发布态的切换

典型接口：

接口用途	代码位置
获取设计信息	<code>GetInfo(long id)</code>
新增模板	<code>Create([FromBody] VisualDevCrInput input)</code>
修改模板	<code>Update(long id, [FromBody] VisualDevUpInput input)</code>
发布模板	<code>FuncToMenu(long id, [FromBody] VisualDevToMenuInput input)</code>
获取发布版模板	<code>GetInfoById(string id, bool isGetRelease = false)</code>

2. VisualDevModelDataService

文件：

- 02-应用模块\10-VisualDev\VisualDev\VisualDevModelDataService.cs

职责：

- 运行时表单数据的增删改查
- 列表分页查询
- 编辑前详情获取
- 规则校验
- 导入导出

典型接口：

接口用途	代码位置
获取编辑详情	<code>GetInfo(string id, string modelId)</code>
新增数据	<code>Create(string modelId, VisualDevModelDataCrInput)</code>
修改数据	<code>Update(string modelId, string id, VisualDevModelDataUpInput)</code>
删除数据	<code>Delete(string id, string modelId)</code>
批量修改	<code>BatchUpdate(string modelId, VisualDevModelDataUpInput)</code>

3. RunService

文件：

- 02-应用模块\10-VisualDev\VisualDev\RunService.cs

职责：

- 根据模板解析实际数据库读写逻辑
- 动态生成新增/修改 SQL
- 处理主表、子表、附表
- 执行事务
- 并发锁、集成助手事件、字段生成等运行时逻辑

这是 表单运行时最核心的执行层。

7. 如果要修改“表单编辑相关接口”，应该从哪里入手？

先区分你要改的是哪一类“编辑”：

7.1 场景一：修改“表单设计器/模板配置”的接口

比如你要改：

- 表单设计页面保存接口
- 表单字段配置的保存结构
- 发布前模板校验
- 设计态 `FormData` / `ColumnData` 的结构

优先从这里入手：

1. 02-应用模块\10-VisualDev\VisualDev\VisualDevService.cs
2. 02-应用模块\10-VisualDev\VisualDev.Entity\Entity\VisualDevEntity.cs
3. 02-应用模块\10-VisualDev\VisualDev.Entity\Entity\VisualDevReleaseEntity.cs
4. 02-应用模块\10-VisualDev\VisualDev.Engine 中的模板解析类

重点看这几个方法：

- `Create(...)`
- `Update(...)`
- `FuncToMenu(...)`
- `GetInfoById(...)`
- `SyncField(...)`

原因：

- 设计器保存时改的是 `BASE_VISUAL_DEV`
- 发布时会把设计态同步到 `BASE_VISUAL_RELEASE`
- 如果只改草稿，不改发布链路，运行时不会生效

7.2 场景二：修改“表单数据编辑”的接口

比如你要改：

- 编辑页回显接口
- 提交编辑接口
- 编辑校验规则
- 编辑时主子表/附表的更新逻辑

建议按下面链路跟：

1. 入口 API

- 02-应用模块\10-VisualDev\VisualDev\VisualDevModelDataService.cs
- 重点方法：`GetInfo(...)`、`Update(...)`

2. 入参模型

- 02-应用模块\00-Common\Common\Dto\VisualDev\VisualDevModelDataUpInput.cs

3. 模板读取

- `IVisualDevService.GetInfoById(modelId, true)`
- 这里默认拿的是发布版模板

4. 规则校验

- `VisualDevModelDataService.CheckRule(...)`

5. 实际更新执行

- `RunService.Update(...)`

6. SQL 生成/表更新

- `RunService.UpdateHaveTableSql(...)`

最重要的实际调用链：

```
VisualDevModelDataService.Update
-> CheckRule
-> IVisualDevService.GetInfoById(modelId, true)
-> RunService.Update
-> UpdateHaveTableSql
-> ExecuteSql
```

7.3 场景三：修改“外链表单”的编辑接口

入口在：

- 02-应用模块\10-VisualDev\VisualDev\VisualDevShortLinkService.cs

这里的更新接口最终也会落到：

- `RunService.Update(...)`

所以：

- 如果只是外链鉴权/解密逻辑变化，看 `VisualDevShortLinkService`
- 如果是通用编辑保存逻辑变化，还是要看 `RunService.Update`

8. 表单编辑相关改造时的实际排查顺序

建议按这个顺序下手：

1. 先确认改的是模板还是数据

- 模板：`VisualDevService`
- 数据：`VisualDevModelDataService`

2. 确认取的是草稿版还是发布版

- `GetInfoById(..., false)`：草稿
- `GetInfoById(..., true)`：发布版

3. 确认入参模型有没有变化

- `VisualDevModelDataCrInput`
- `VisualDevModelDataUpInput`

4. 确认校验规则是否会受影响

- `CheckRule(...)`

5. 确认最终 SQL 生成逻辑是否要改

- `RunService.Update(...)`
- `RunService.UpdateHaveTableSql(...)`

6. 确认发布链路是否同步

- 如果改了 `FormData` 结构，通常还要检查发布态实体与发布逻辑

9. 建议的学习顺序（针对后续要改表单编辑接口）

如果你的目标就是后续改表单编辑接口，建议直接按这个顺序读：

1. 03-应用服务\IotPlatform\Program.cs
2. 03-应用服务\IotPlatform.Web.Core\Startup.cs
3. 02-应用模块\10-VisualDev\VisualDev.Entity\Entity\VisualDevEntity.cs
4. 02-应用模块\10-VisualDev\VisualDev.Entity\Entity\VisualDevReleaseEntity.cs
5. 02-应用模块\10-VisualDev\VisualDev\VisualDevService.cs
6. 02-应用模块\10-VisualDev\VisualDev\VisualDevModelDataService.cs
7. 02-应用模块\10-VisualDev\VisualDev\RunService.cs
8. 02-应用模块\00-Common\Common\Dto\VisualDev\VisualDevModelDataUpInput.cs
9. 02-应用模块\10-VisualDev\VisualDev.Interface\IRunService.cs

读完这 9 个文件，基本就能把：

- 模板如何保存
- 模板如何发布
- 运行时如何读取发布版模板
- 编辑数据如何校验
- 编辑数据如何落库

这 5 条主线串起来。

10. 总结

这个项目最值得先抓住的不是某一个接口，而是两条主线：

1. 平台主线： `Program.cs` / `Startup.cs` / 各业务模块动态 API
2. 表单主线： `VisualDevEntity` -> `VisualDevService` -> `VisualDevModelDataService` -> `RunService`

如果后续要改“表单编辑相关接口”，首选从 `VisualDevModelDataService.Update` 和 `RunService.Update` 开始追；如果改的是设计器保存能力，则从 `VisualDevService.Update` 开始。